

DATA TYPE

BY SUJAL SHRESTHA

Data Type

- Data Type represent the type of data present inside a variable.
- In Python we are not required to specify the type explicitly. Based on value provided,the type will be assigned automatically. Hence Python is Dynamically Typed Language.
- Python contains the following in-built data types:
int, float, complex, bool, str, bytes, bytearray, range, list, tuple, set, frozenset, dict and None.

Data Type

Every thing is an object in Python.

Example, name = "Sujal Shrestha".

In above, "Sujal Shrestha" is an object of class 'string'.

Here, name is the reference variable which is pointing to 'string' object.

The value present in the 'string' object is "Sujal Shrestha".

type(), id() and print()

- `type()`:

By using an in-built function `type()`, we can find the type of any variable.

```
name = "Sujal Shrestha"
```

```
type(name)
```

- `id()`

By using an in-built function `id()`, we can find the address of an object.

```
name = "Sujal Shrestha"
```

```
id(name)
```

- `print()`

By using in-built function `print()`, we can print the value.

```
print("I love Kathmandu")
```

Data Type

Which of the following statements is correct regarding Python?

1. Python does not use the concept of data types.
2. Python uses data types, but we do not need to declare them explicitly.

int

In Python, int is a built-in data type used to represent whole numbers (integral values). It is one of the most commonly used types in Python programming for mathematical operations, counting, indexing, and slicing.

```
a=20
```

```
b="20"
```

```
int(b)
```

Note: Strings or floating-point numbers can be converted to integers using the int() function.

int

Is 782938465000029742852673894950222 an integer?

int

We can represent integral values in the following ways:

1. Decimal form (base 10)
2. Binary form (base 2)
3. Octal form (base 8)
4. Hexa decimal form (base 16)

int

By default, integers are base-10 (decimal). Integer value can be changed to other bases, such as binary, octal, or hexadecimal, using prefixes:

- Binary: 0b (e.g., 0b1010)
- Octal: 0o (e.g., 0o12)
- Hexadecimal: 0x (e.g., 0xA)

```
print("Binary value representation:", 0b1010)
print("Octal value representation:", 0o12)
print("Hexadecimal value representation:", 0xa)
```

Output

```
Binary value representation: 10
Octal value representation: 10
Hexadecimal value representation: 10
```

int

Data type base conversion functions.

- `bin()`: It converts any other bases to binary
- `oct()`: It converts any other bases to octal
- `hex()`: It converts any other bases to hexadecimal

```
1 print("dec, oct and hex to binary:", bin(10), bin(0o12), bin(0xA))
2 print("bin, dec, hex to octal:", oct(0b1010), oct(10), oct(0xA))
3 print("bin, oct, dec to hex:", hex(0b1010), hex(0o12), hex(10))
4
```

Output

```
dec, oct and hex to binary: 0b1010 0b1010 0b1010
bin, dec, hex to octal: 0o12 0o12 0o12
bin, oct, dec to hex: 0xa 0xa 0xa
```

int

int() is used to convert other data types like float, string, bool to integer.

```
1 num_f=3.99
2 num_s="10"
3 num_b=True
4 print(int(num_f), int(num_s), int(num_b))
```

Output

```
3 10 1
```

Float

In Python, the float data type represents floating-point numbers, which are numbers with a decimal point. It is used to perform calculations that require more precision compared to integers.

Example

```
pi=3.14
```

Note: We can represent integral values in decimal, binary, octal and hexa decimal forms. But we can represent float values only by using decimal form.

Float

We can represent floating point values by using exponential form (scientific notation). In Python, float exponential notation (also called scientific notation) is a way to represent very large or very small floating-point numbers using powers of 10. The main advantage of exponential form is we can represent big values in less memory.

```
1 num=3.23e2
2 print(num)
3 num1= 1.4E3
4 print(num1)
5
```

Output

323.0

1400.0

Float

We can use `float()` is used to convert other data types like int, string, bool to floating-point numbers.

```
1 num=10
2 num_s="3.14"
3 num_b=False
4 print(float(num), float(num_s), float(num_b))
```

Output

```
10.0 3.14 0.0
```

Bool

The bool type in Python is used to represent Boolean values: True or False. The only allowed values for this data type are: True and False (true & false are not allowed in Python) Internally Python represents True as 1 and False as 0.

Output	
1 print(True + True)	2
2 print(True-False)	1
3 print(True * False)	0

Bool

The bool type in Python is used to represent Boolean values: True or False. The only allowed values for this data type are: True and False (true & false are not allowed in Python) Internally Python represents True as 1 and False as 0.

```
1 print(bool(1), bool(0.0), bool("Hi"), bool(""))  
2
```

Output

True False True False

Complex

Complex data type is used to represent complex numbers. A complex number consist a real part and an imaginary part that can be written as $z=a+bj$.

Here, a = real part

b = imaginary part

j = imaginary unit (square root of -1)

z = complex number

```
1 com= 4+5j
2 print(com, type(com), com.real, com.imag)
```

Output

```
(4+5j) <class 'complex'> 4.0 5.0
```

Complex

In complex data type, we can perform mathematical operations like $+$, $-$, $/$ and $\times$.

```
1 c1=2+3j
2 c2=5+6j
3 print(c1+c2)
4 print(c1-c2)
```

Output

```
(7+9j)
(-3-3j)
```

Complex

To represent complex number, imaginary part should be mentioned compulsory.

```
1 com=5j
2 real=5
3 print(com, type(com))
4 print(real, type(real))
```

Output

```
5j <class 'complex'>
5 <class 'int'>
```

Type Casting in Python

Type Casting is the method to convert the python variable datatype into certain data type. The two types of Type Casting in Python are:

- Implicit Type Casting
- Explicit Type Casting

Type Casting in Python

Implicit Type Casting

In Implicit Type Casting, Python converts the data type into another datatype automatically.

```
1 num = 10
2 num1 = 3
3 print(num/num1, type(num/num1))
```

Output

```
3.3333333333333335 <class 'float'>
```


Type Casting in Python

Explicit Type Casting

In Explicit Type Casting, Python needs user involvement to convert the variable data type into the required data type. Eg, `int()`, `str()`, `float()`

```
1 num=3.99
2 print(num, type(num))
3 num1=int(num)
4 print(num1, type(num1))
```

Output

```
3.99 <class 'float'>
3 <class 'int'>
```

input ()

In Python, the input() function is used to take user input.

1 Basic Input

Takes input from user as a string by default.

```
1 name=input("Enter your Name: ")
2 print(name, type(name))
Enter your Name: Harry Potter
Harry Potter <class 'str'>
```

2 Input with Type Casting

Here, input is explicitly converted into other data type.

```
1 age=int(input("Enter your Age: "))
2 print(age, type(age))
```

```
Enter your Age: 16
16 <class 'int'>
```

input ()

3 Multiple Inputs on One Line

In order to take multiple input from user, we need number of variables for input and need to use `".split()"`. Here, `".split()"` divides the string by space or any delimiter.

```
1 f, f1, f2 = input("Enter fruits name: ").split()  
2 print(f, f1, f2)  
3 print(type(f), type(f1), type(f2))
```

```
Enter fruits name: Hydrogen Nitrogen Oxygen  
Hydrogen Nitrogen Oxygen  
<class 'str'> <class 'str'> <class 'str'>
```


String Formatting

String formatting in Python is the process of inserting variables or expressions into strings in a structured, readable, and often styled way using the `print()` function.

The ways to format output in python:

1. Using `%` operator
2. Using `str.format()` method
3. Using f-Strings

String Formatting

Using % Operator

A traditional C-style method where placeholders are replaced with variable values.

Syntax:

"format_string"%values

Format Specifiers:

%s → String

%d → Integer

%f → Floating point

```
3  ward=13
4  address="Kathmandu %d"%ward
5  print("I live in %s"%address)
```

Output

I live in Kathmandu 13

String Formatting

Using str.format() method

This method allows more control and flexibility using {} placeholders.

Syntax:

"format string {}".format(values)

```
3 ward=13
4 address="Kathmandu {}".format(ward)
5 print("I live in {}".format(address))
```

Output

I live in Kathmandu 13

String Formatting

f-string

This process is modern and preferred way to embed expressions inside strings using {} prefixed by f.

Syntax:

f"some text {expression or variable}"

```
3 ward=13
4 address=f"Kathmandu {ward}"
5 print(f"I live in {address}")
```

Output

I live in Kathmandu 13

String Formatting

Using str.format() method

This process is modern and preferred way to embed expressions inside strings using {} prefixed by f.

Syntax:

f"some text {expression or variable}"

```
3 ward=13
4 address=f"Kathmandu {ward}"
5 print(f"I live in {address}")
```

Output

I live in Kathmandu 13

eval()

eval() Function is a single function which is the replacement of all the typecasting functions in Python. Using eval() we can evaluate like string mathematical expressions entered by the user.

main.py

```
1 var = eval(input("Enter Anything: "))  
2 print(var, type(var))
```

Output

Enter Anything: "Sujal Shrestha"
Sujal Shrestha <class 'str'>

Output

Enter Anything: 100500
100500 <class 'int'>

Output

Enter Anything: 3.14
3.14 <class 'float'>

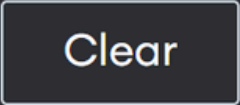
strip()

strip() removes spaces from the beginning and end of a string.

main.py					Run	Output
1	name = " Sandip Lamichhane "					Sandip Lamichhane
2	print(name)					Sandip Lamichhane
3	print(name.strip())					
						=== Code Execution Successful ===

split()

split() divides a string into multiple pieces based on a separator (default: space).

main.py	   	Output 
<pre>1 fruit = "Apple Banana Cherry".split() 2 print(fruit) 3 name, age, address = input("Enter name, age, address: ").split(",") 4 print(f"Name: {name}, Age: {age}, Address: {address}")</pre>	<pre>['Apple', 'Banana', 'Cherry'] Enter name, age, address: Sandesh Raj Thakur, 24, Lahan Name: Sandesh Raj Thakur, Age: 24, Address: Lahan === Code Execution Successful ===</pre>	

Command Line Argument

Command Line Arguments are extra values that you give to a Python program when you run it from the Command Prompt (CMD).

```
1 # Finding Area of Rectangle using Command Line Arguments
2 import sys
3
4 print("Program name:", sys.argv[0])
5
6 length = int(sys.argv[1])
7 breadth = int(sys.argv[2])
8 area = length * breadth
9 print("Area of Rectangle:", area)
```

▼ TERMINAL

- PS E:\Documents\Python> cd MyPractice
- PS E:\Documents\Python\MyPractice> python first.py 5 5

Program name: first.py

Area of Rectangle: 25

Concept	Meaning
Command Line Arguments	Values given to a program during execution
sys module	Helps Python read arguments
sys.argv	Stores arguments as a list
sys.argv[0]	Program name
sys.argv[1], sys.argv[2], ...	User-given inputs

range()

Commonly used in for loops to run a block of code multiple times.

```
1 a= range(10)
2 print(a, type(a))
```

range(0, 10) <class 'range'>

=== Code Execution Successful ===

Example: write a python programme to print the number from 0 to 4.

```
1 for i in range(5):
2     print(i)
```

0
1
2
3
4

=== Code Execution Successful ===

range()

range Data Type represents a sequence of numbers. range() is the in-built function of Python. The elements present in range Data type are not modifiable. i.e range Data type is immutable.

Syntax:

range(stop)

range(start, stop)

range(start, stop, step)

range()

Different forms of range data type are as follows:

1. range with one argument - `range(10)`
2. range with Two arguments - `range(10, 21)`
3. range with three arguments - `range(10, 21, 2)`

STRING IN PYTHON

A string in Python is a sequence of characters enclosed in single quotes, double quotes, or triple quotes. Strings are immutable, meaning their contents cannot be changed once they are created.

String is enclosed in quotes:

- Single quotes: 'Hello'
- Double quotes: "Hello"
- Triple quotes (used for multi-line strings): '''Hello''' or """Hello"""

CONCATENATION AND REPETITION IN STRING

Strings can be concatenated using + and repeated using *.

Example:

```
s1 = "Hello"
```

```
s2 = "World"
```

```
print(s1 + " " + s2) # Output: 'Hello World'
```

```
print(s1 * 3) # Output: 'HelloHelloHello'
```

ACCESS CHARACTERS OF A STRING

a. By using index:

Python supports both +ve and -ve index.

+ve index means left to right(Forward direction)

-ve index means right to left(Backward direction)

Example:

```
s = "Python"
```

```
print(s[0])
```

```
print(s[-1])
```

ACCESS CHARACTERS OF A STRING

b. Accessing characters by using slice operator:

String slice means a part of the string (i.e, Sub string).

Syntax:

`string_Name [beginindex:endindex:step]`

beginindex: From where we have to consider slice(substring)

endindex: We have to terminate the slice(substring) at endindex-1

step: incremented / decremented value.

ACCESS CHARACTERS OF A STRING

In forward direction:

- default value for begin: 0

- default value for end: length of string

- default value for step: +1

In backward direction:

- default value for begin: -1

- default value for end: $-(\text{length of string} + 1)$

Note:

In the backward direction if end value is -1 then result is always empty.

In the forward direction if end value is 0 then result is always empty.

STRING METHODS IN PYTHON

i. Case Conversion

`str.upper()`

`str.lower()`

`str.title()`

`str.capitalize()`

`str.swapcase()`

STRING METHODS IN PYTHON

ii. Searching and Replacing

`str.find(sub)` #returns the index of first occurrence of sub, or -1 if not found

`str.index(sub)` #similar to find(), but raises error if not found

`str.count(sub)` #returns the number of occurrence of sub

`str.replace(old, new)` #replace all occurrence of old with new

STRING METHODS IN PYTHON

iii. Checking String Properties

`str.isalpha()` #returns True if the string only alphabetic characters

`str.isdigit()` #returns True if the string only digits

`str.isalnum()` # returns True if the string contains only alphanumeric characters

`str.isspace()` #returns True if the string only contains whitespace characters

`str.startswith(prefix)` #checks the string started with

`str.endswith(suffix)` #checks the string ended with

len() function in python:

In Python, the len() function is used to return the number of items (elements or characters) in an object like a string, list, tuple, dictionary, or any iterable.

Syntax:

```
len(object)
```

Example:

```
actor= "Daya Hang Rai"  
print(len(actor)) #13
```


QUESTIONS

1. Write a Python program to
 - a. Display "I am a big fan of Kutumba band. Their traditional music connect generations."
 - b. Count the length of this string.
 - c. Slice this string up to "I am a big fan of Kutumba band."
 - d. Find the word "traditional" and replace with "folk".
 - e. Concatenate " because their instruments are soulful and INSPIRING!".
 - f. Display "Kutumba" in upper case
 - g. Display "INSPIRING" in lower case
 - h. Count letter "a" present in the final concatenated string.

QUESTIONS

- Write a python programme to take full name input from user and reverse it using slicing concept.
- Convert the username into upper case and lower case using string function.
- Take a username input and print the first name and last name slice shown in the example below:
First Name: Balendra
Last Name: Shah
- Print the odd and even indices of the user input using slicing.
- Find the length of the user input.

LIST

A list is a versatile and mutable data structure in Python that is used to store multiple items in a single variable. Items in a list are ordered, changeable, and allow duplicate values.

Syntax:

```
list_name = [item1, item2, item3, ...]
```

Example:

```
list1=["apple", "banana", "cat"]
```


TUPLE

A tuple in Python is an immutable, ordered collection of items. Tuples are similar to lists, but once created, their elements cannot be changed, added, or removed.

They are useful for storing fixed collections of items, such as coordinates, RGB color values, or constant data.

Syntax:

```
my_tuple = ( item1, item2, item3 )  
print(my_tuple)
```

SET

A set in Python is an unordered collection of unique and mutable elements. Sets are commonly used to store elements where duplicates are not allowed, and their operations are optimized for checking membership, union, intersection, and difference.

Syntax:

```
myset={item1, item2, item3}  
print(myset)
```

PYTHON DICTIONARY

A dictionary in Python is a built-in data type that allows you to store and manipulate data in key-value pairs. It is mutable, unordered as of Python 3.6 (though it maintains insertion order in Python 3.7 and later), and can store heterogeneous data types.

Dictionary Syntax:

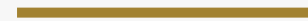
Creating a dictionary

```
my_dict = { "key1": "value1", "key2": "value2", "key3": "value3" }
```

Accessing values

```
print(my_dict["key1"])
```

THANK YOU



FOR COMING